DAY 2 – PYTHON OPERATORS, INPUT & CONDITIONAL STATEMENTS

1. What Are Operators?

Operators are symbols that tell Python to perform some operation.

For example:

```
a = 10
b = 5

total = a + b

print(total)
```

Output

15

Here:

a → Variable

10 → Value

b → Variable

5 → Value

\+ → Operator

total → Variable storing the result

Python takes 10 + 5 and stores the result 15 inside total.

---

2. Arithmetic Operators

Arithmetic operators are used for mathematical calculations.

\+ Addition

\- Subtraction

* Multiplication

/ Division

% Modulus

// Floor Division

** Power

Example:

```
a = 10
b = 3

print(a + b)
print(a - b)
print(a * b)
print(a / b)
print(a % b)
print(a // b)
print(a ** b)
```

Output

13

7

30

3.3333333333333335

1

3

1000

Let's understand the important ones.

print(10 % 3)

Output:

1

% gives the **remainder**.

Because:

10 / 3

3 × 3 = 9

Remaining = 1

Floor division:

print(10 // 3)

Output:

3

Power:

print(2 ** 3)

Output:

8

Because:

2 × 2 × 2 = 8

---

3. Data Engineering Example – Calculate Total Records

Suppose we processed records from three files.

```
customers = 1000
orders = 2500
products = 500

total_records = customers + orders + products

print("Total Records:", total_records)
```

Output

Total Records: 4000

This is a simple example of how operators can be used in a data pipeline.

---

4. Comparison Operators

Comparison operators compare two values.

The result will normally be:

True

or:

False

Important comparison operators:

== Equal to

!= Not equal to

> Greater than

< Less than

>= Greater than or equal to

<= Less than or equal to

Example:

```
a = 10
b = 5

print(a > b)
```

```
print(a < b)
print(a == b)
print(a != b)
```

Output

True

False

False

True

Why?

10 > 5 → True

10 < 5 → False

10 == 5 → False

10 != 5 → True

---

5. = vs ==

Students should understand this clearly.

status = "SUCCESS"

Here:

= → Assignment

We are storing "SUCCESS" inside status.

But:

status == "SUCCESS"

means:

Is the value inside status equal to "SUCCESS"?

Example:

```
status = "SUCCESS"

print(status == "SUCCESS")
```

Output

True

So remember:

= Store a value

== Compare two values

---

6. Logical Operators

The main logical operators are:

and

or

not

and

Both conditions should be true.

```
status = "SUCCESS"
records = 1000

print(status == "SUCCESS" and records > 500)
```

Output

True

Why?

status == "SUCCESS" → True

records > 500 → True

Both are true, so the final answer is:

True

---

7. or Operator

For or, at least one condition needs to be true.

```
source = "S3"

print(source == "S3" or source == "RDS")
```

Output

True

The first condition is already true.

---

8. not Operator

not reverses the result.

```
status = "FAILED"

print(not status == "SUCCESS")
```

Output

True

Because:

status == "SUCCESS"

is false.

not changes false to true.

---

9. Assignment Operators

We already know:

records = 1000

We can increase it:

```
records = 1000

records = records + 500

print(records)
```

Output

1500

Python provides a shorter way:

```
records = 1000

records += 500

print(records)
```

Output

1500

Similarly:

+=

-=

*=

/=

---

10. User Input

Until now we manually provided values.

Example:

name = "Arun"

But sometimes we want the user to enter the value.

For that we use:

input()

Example:

```
name = input("Enter your name: ")

print("Hello", name)
```

Suppose the user enters:

Arun

Output

Enter your name: Arun

Hello Arun

---

11. Data Engineering Input Example

```
file_name = input("Enter file name: ")

print("Processing file:", file_name)
```

Suppose the user enters:

customers.csv

Output

Enter file name: customers.csv

Processing file: customers.csv

---

12. Important – input() Returns String

Look at:

```
records = input("Enter record count: ")

print(type(records))
```

Suppose we enter:

1000

Output

<class 'str'>

Even though we entered 1000, Python treats input as a **string**.

Therefore, we need type conversion.

---

13. Type Conversion

Convert string to integer:

```
records = int(input("Enter record count: "))

print(records)
print(type(records))
```

Input:

1000

Output

1000

<class 'int'>

Common conversions:

int() → Integer

float() → Decimal number

str() → String

bool() → Boolean

Example:

```
value = "1000"

records = int(value)

print(records)
print(type(records))
```

Output

1000

<class 'int'>

---

14. Conditional Statements

Now we come to one of the most important Python concepts.

Conditional statements allow our program to **make decisions**.

For example:

Is file successful?

YES → Process file

NO → Don't process file

In Python we use:

if

elif

else

---

15. Simple if Condition

```
records = 1000

if records > 500:
    print("Large number of records")
```

Output

Large number of records

Why?

Python checks:

1000 > 500

Result:

True

Therefore Python executes:

print("Large number of records")

---

16. Indentation Is Important

Look carefully:

```
if records > 500:
    print("Large file")
```

The spaces before print() are called **indentation**.

Correct:

```
if records > 500:
    print("Large file")
```

Incorrect:

```
if records > 500:
print("Large file")
```

Python uses indentation to understand which code belongs inside the condition.

---

17. if-else

Suppose we need two possibilities.

```
status = "SUCCESS"

if status == "SUCCESS":
    print("Process the file")
else:
    print("Do not process the file")
```

Output

Process the file

Flow:

status == SUCCESS?

|

| |

YES NO

| |

Process Don't

file process

---

18. Failed File Example

```
status = "FAILED"

if status == "SUCCESS":
    print("Process the file")
else:
    print("Do not process the file")
```

Output

Do not process the file

---

19. if-elif-else

Sometimes we have more than two conditions.

Example:

```
records = 5000

if records > 10000:
    print("Large file")
elif records > 1000:
    print("Medium file")
else:
    print("Small file")
```

Output

Medium file

Python checks from top to bottom.

First:

5000 > 10000 → False

Next:

5000 > 1000 → True

So Python prints:

Medium file

---

20. Multiple Conditions

Now combine conditions using and.

```
status = "SUCCESS"
records = 5000

if status == "SUCCESS" and records > 1000:
    print("Start processing")
else:
    print("Cannot process")
```

Output

Start processing

Both conditions are true.

---

21. AWS Data Engineering Scenario

Imagine an S3 file arrives.

We want to process the file only when:

Status = SUCCESS

AND

File size < 500 MB

Program:

```
file_name = "customers.csv"
file_status = "SUCCESS"
file_size = 120

if file_status == "SUCCESS" and file_size < 500:
    print("File:", file_name)
    print("File is ready for processing")
else:
    print("File cannot be processed")
```

Output

File: customers.csv

File is ready for processing

This is how basic Python conditions start connecting with real Data Engineering logic.

---

22. Practice Program 1

```
file_name = input("Enter file name: ")
record_count = int(input("Enter record count: "))

if record_count > 1000:
    print(file_name, "is a large file")
else:
    print(file_name, "is a small file")
```

Input:

customers.csv

5000

Expected Output

customers.csv is a large file

---

23. Practice Program 2

```
pipeline_status = input("Enter pipeline status: ")

if pipeline_status == "SUCCESS":
    print("Pipeline completed successfully")
else:
    print("Pipeline failed")
```

Input:

SUCCESS

Expected Output

Pipeline completed successfully

---

24. Day 2 Summary

Students should understand:

Operators

↓

Arithmetic Operators

↓

Comparison Operators

↓

Logical Operators

↓

Assignment Operators

↓

input()

↓

Type Conversion

↓

if

↓

if-else

↓

if-elif-else

↓

Multiple Conditions

↓

Data Engineering Conditions

DAY 4 – PYTHON LOOPS FOR AWS DATA ENGINEERING

After Day 3 collections, Day 4 should focus strongly on **loops**.

In Data Engineering, we don't normally process only one file or one record.

We may process:

100 files

1,000 files

10,000 files

Millions of records

We cannot manually write code for every file.

That's why we use loops.

---

1. What Is a Loop?

Suppose we have:

customers.csv

orders.csv

products.csv

Without a loop:

```
print("Processing customers.csv")
print("Processing orders.csv")
print("Processing products.csv")
```

Output

Processing customers.csv

Processing orders.csv

Processing products.csv

This works for three files.

But what if we have 1,000 files?

We use a **loop**.

---

2. for Loop

Example:

```
files = [
    "customers.csv",
    "orders.csv",
    "products.csv"
]

for file in files:
    print(file)
```

Output

customers.csv

orders.csv

products.csv

Understand this carefully.

for file in files:

means:

Take one value at a time from files and temporarily store that value inside file.

---

3. How the Loop Works

Our list:

```
files = [
    "customers.csv",
    "orders.csv",
    "products.csv"
]
```

First iteration:

file = customers.csv

Python prints:

customers.csv

Second iteration:

file = orders.csv

Third iteration:

file = products.csv

Then the loop ends.

---

4. Data Engineering Example

```
files = [
    "customers.csv",
    "orders.csv",
    "products.csv"
]

for file in files:
    print("Processing:", file)
```

Output

Processing: customers.csv

Processing: orders.csv

Processing: products.csv

This is very close to what we do when processing multiple S3 objects.

---

5. Loop Through Numbers

```
numbers = [10, 20, 30, 40]

for number in numbers:
    print(number)
```

Output

10

20

30

40

---

6. range()

range() is used when we want to repeat something a certain number of times.

```
for number in range(5):
    print(number)
```

Output

0

1

2

3

4

Notice:

range(5)

starts from 0 and stops **before 5**.

---

7. range(start, stop)

```
for number in range(1, 6):
    print(number)
```

Output

1

2

3

4

5

Here:

Start = 1

Stop = 6

Python stops before 6.

---

8. range(start, stop, step)

```
for number in range(0, 10, 2):
    print(number)
```

Output

0

2

4

6

8

Here:

0 → Start

10 → Stop

2 → Step

So Python increases the value by 2 each time.

---

9. Process Files with File Number

```
for file_number in range(1, 6):
    print("Processing file:", file_number)
```

Output

Processing file: 1

Processing file: 2

Processing file: 3

Processing file: 4

Processing file: 5

---

10. Loop with if Condition

We can combine loops and conditions.

```
files = [
    "customers.csv",
    "orders.txt",
    "products.csv"
]

for file in files:
    if file.endswith(".csv"):
        print("Processing:", file)
```

Output

Processing: customers.csv

Processing: products.csv

Why wasn't orders.txt processed?

Because:

file.endswith(".csv")

is false for orders.txt.

This is a very useful Data Engineering concept.

---

11. Loop Through Dictionary

Example:

```
customer = {
    "id": 101,
    "name": "Arun",
    "city": "Chennai"
}

for key in customer:
    print(key)
```

Output

id

name

city

---

12. Dictionary Keys and Values

To get both:

```
customer = {
    "id": 101,
    "name": "Arun",
    "city": "Chennai"
}

for key, value in customer.items():
    print(key, ":", value)
```

Output

id : 101

name : Arun

city : Chennai

This is important when processing JSON-like data.

---

13. Loop Through List of Dictionaries

From Day 3 we learned:

```
customers = [
    {"id": 101, "name": "Arun"},
    {"id": 102, "name": "Priya"},
    {"id": 103, "name": "Kumar"}
]
```

Now loop through it:

```
for customer in customers:
    print(customer["id"], customer["name"])
```

Output

101 Arun

102 Priya

103 Kumar

This combines:

Day 3 → List + Dictionary

Day 4 → Loop

This is why the learning sequence is important.

---

14. while Loop

A while loop runs as long as a condition is true.

Example:

```
count = 1

while count <= 5:
    print(count)
    count = count + 1
```

Output

1

2

3

4

5

Let's understand it.

Initially:

count = 1

Check:

1 <= 5 → True

Print 1.

Then:

count = count + 1

Now:

count = 2

This continues until count becomes 6.

Then:

6 <= 5 → False

The loop stops.

---

15. Why Increment Is Important

Be careful with:

```
count = 1

while count <= 5:
    print(count)
```

There is no:

count = count + 1

So count always remains 1.

The condition always remains true.

This creates an **infinite loop**.

Correct:

```
count = 1

while count <= 5:
    print(count)
    count += 1
```

---

16. Data Engineering Retry Example

A simple retry concept:

```
attempt = 1

while attempt <= 3:
    print("Pipeline attempt:", attempt)
    attempt += 1
```

Output

Pipeline attempt: 1

Pipeline attempt: 2

Pipeline attempt: 3

Later, this concept can be connected to retrying an operation when an API or another service temporarily fails.

---

17. break

break immediately stops a loop.

Example:

```
for number in range(1, 10):
    if number == 5:
        break

    print(number)
```

Output

1

2

3

4

When Python reaches:

number = 5

this condition becomes true:

number == 5

Then:

break

stops the entire loop.

---

18. Data Engineering Example with break

```
files = [
    "customers.csv",
    "orders.csv",
    "STOP",
    "products.csv"
]

for file in files:
    if file == "STOP":
        break

    print("Processing:", file)
```

Output

Processing: customers.csv

Processing: orders.csv

Once Python sees STOP, the loop ends.

Therefore:

products.csv

is not processed.

---

19. continue

continue is different from break.

break:

Stop the entire loop

continue:

Skip only the current item and continue with the next item

Example:

```
for number in range(1, 6):
    if number == 3:
        continue

    print(number)
```

Output

1

2

4

5

3 is skipped, but the loop continues.

---

20. Data Engineering Example with continue

Suppose we don't want to process .txt files.

```
files = [
    "customers.csv",
    "notes.txt",
    "orders.csv",
    "readme.txt",
    "products.csv"
]

for file in files:
    if file.endswith(".txt"):
        continue

    print("Processing:", file)
```

Output

Processing: customers.csv

Processing: orders.csv

Processing: products.csv

This is a useful file-processing pattern.

---

21. break vs continue

Remember:

break

↓

Stop the complete loop

continue

↓

Skip current item

↓

Continue with next item

Example:

```
files = [
    "customers.csv",
    "bad_file.csv",
    "orders.csv"
]

for file in files:
    if file == "bad_file.csv":
        continue

    print("Processing:", file)
```

Output

Processing: customers.csv

Processing: orders.csv

---

22. Nested Loop

A loop inside another loop is called a **nested loop**.

Example:

```
regions = ["India", "USA"]
files = ["customers.csv", "orders.csv"]

for region in regions:
    for file in files:
        print(region, "-", file)
```

Output

India - customers.csv

India - orders.csv

USA - customers.csv

USA - orders.csv

First Python takes:

India

Then processes all files.

After that it takes:

USA

and processes all files again.

---

23. AWS Data Engineering Nested Loop Example

Imagine we have multiple S3 folders and multiple files.

```
folders = [
    "customers",
    "orders"
]

dates = [
    "2026-10-01",
    "2026-10-02"
]

for folder in folders:
    for date in dates:
        print("Processing:", folder, date)
```

Output

Processing: customers 2026-10-01

Processing: customers 2026-10-02

Processing: orders 2026-10-01

Processing: orders 2026-10-02

This helps students understand how loops can be used when processing data across folders, dates, partitions, or other groups.

---

24. Real Data Engineering Scenario

Suppose our pipeline receives this:

```
files = [
    {
        "file_name": "customers.csv",
        "status": "SUCCESS",
        "records": 1000
    },
    {
        "file_name": "orders.csv",
        "status": "FAILED",
        "records": 0
    },
    {
        "file_name": "products.csv",
        "status": "SUCCESS",
        "records": 500
    }
]
```

We want to process only successful files.

```
for file in files:
    if file["status"] == "FAILED":
```

```
        print("Skipping:", file["file_name"])
        continue

    print(
        "Processing:",
        file["file_name"],
        "- Records:",
        file["records"]
    )
```

Output

Processing: customers.csv - Records: 1000

Skipping: orders.csv

Processing: products.csv - Records: 500

This one example combines everything:

List

↓

Dictionary

↓

for loop

↓

if condition

↓

continue

↓

File processing

---

25. Count Successfully Processed Files

Now let's make it slightly more practical.

```
files = [
    {"name": "customers.csv", "status": "SUCCESS"},
    {"name": "orders.csv", "status": "FAILED"},
    {"name": "products.csv", "status": "SUCCESS"},
    {"name": "sales.csv", "status": "SUCCESS"}
]

success_count = 0

for file in files:
    if file["status"] == "SUCCESS":
        print("Processing:", file["name"])
        success_count += 1
```

```
print("Successfully processed files:", success_count)
```

Output

Processing: customers.csv

Processing: products.csv

Processing: sales.csv

Successfully processed files: 3

Here:

success_count = 0

starts the count from zero.

Every time we find a successful file:

success_count += 1

adds 1.

Finally:

success_count = 3

---

26. Practice Program 1 – Process Files

```
files = [
    "customers.csv",
    "orders.csv",
    "products.csv",
    "sales.csv"
]

for file in files:
    print("Processing:", file)
```

Expected Output

Processing: customers.csv

Processing: orders.csv

Processing: products.csv

Processing: sales.csv

---

27. Practice Program 2 – Process Only CSV Files

```
files = [
    "customers.csv",
    "notes.txt",
    "orders.csv",
    "readme.txt"
```

```
]

for file in files:
    if file.endswith(".txt"):
        continue

    print("Processing:", file)
```

Expected Output

Processing: customers.csv

Processing: orders.csv

---

28. Practice Program 3 – Pipeline Status

```
pipelines = [
    {"name": "customer_etl", "status": "SUCCESS"},
    {"name": "order_etl", "status": "FAILED"},
    {"name": "product_etl", "status": "SUCCESS"}
]

for pipeline in pipelines:
    if pipeline["status"] == "SUCCESS":
        print(pipeline["name"], "- Completed")
    else:
        print(pipeline["name"], "- Failed")
```

Expected Output

customer_etl - Completed

order_etl - Failed

product_etl - Completed

---

29. Practice Program 4 – Retry

```
attempt = 1

while attempt <= 3:
    print("Trying pipeline - Attempt:", attempt)
    attempt += 1
```

Expected Output

Trying pipeline - Attempt: 1

Trying pipeline - Attempt: 2

Trying pipeline - Attempt: 3

---

30. Day 4 Summary

Students should be comfortable with:

for loop

↓

Loop through list

↓

range()

↓

Loop + if condition

↓

Loop through dictionary

↓

List of dictionaries

↓

while loop

↓

break

↓

continue

↓

Nested loops

↓

AWS/Data Engineering file-processing scenarios

After these four days, the sequence is now properly connected:

Day 1: Python Fundamentals → Variables → Data Types
Day 2: Operators → Input → Type Conversion → Conditions
Day 3: Lists → Tuples → Sets → Dictionaries → List of Dictionaries
Day 4: for → range() → while → break → continue → Nested Loops → Data Engineering processing